



Проследяване и контрол върху качество на изпълнение на проекта.
Оценяване на софтуерни проекти. Прагматични, теоретично-базирани, регресионно-базирани форми (модели) за оценяване.
Инструменти за оценяване на софтуерни проекти: Delphi estimation, WBS/CPM/PERT, SLIM, COCOMO модел, Monte-Carlo модел.

Проследяване и
контрол върху
качеството на
изпълнение на
проекта

01

Оценяване на
софтуерни проекти

02

Прагматични
форми(модели) на
оценяване

03

Какво ще видим в презентацията

04

Теоретично-
базирани
форми(модели) на
оценяване

05

Регресивно-базирани
форми(модели) на
оценяване

06

Инструменти за
оценяване

Delphi estimation,
WBS/CPM/PERT, SLIM, COCOMO
модел, Monte-Carlo модел.





01

Проследяване и контрол върху качество на изпълнение
на проекта



-
1. Какво представлява качеството?
 2. Кой носи отговорност за качеството?
-



Проследяване на качество по изпълнение на проекта

- Какво е мониторинг?
- За какво е предназначено проследяването на проекта?
- Какви са индикаторите за проследяване на качеството по изпълнение на проекта?
- Каква организация и технология е необходима?







02

Оценяване на софтуерни проекти

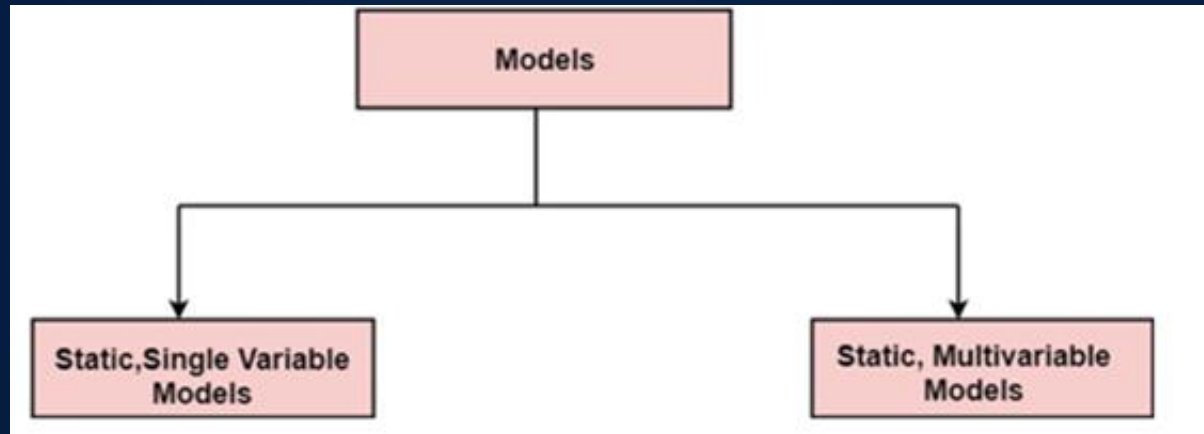


Защо ни трябва оценката на софтуерния продукт





Модели за оценка (статични и динамични)





03

Прагматични форми на оценяване



Какво
представлява добра
оценка?

**проблем при оценката на
разходите за софтуер**

Измервате размера на
софтуера в изходните
линии на кода или
функционалните точки?



Кой модел за оценка
на разходите да се
използва ?

Добра оценка



- Той е замислен и подкрепен от ръководителя на проекта, архитектурния екип, екипа по разработка и тестовия екип, отговорен за изпълнението на работата.
- Приет е от всички заинтересовани страни като амбициозен, но осъществим.
- Той се основава на добре дефиниран модел на разходите за софтуер с надеждна основа.
- Основава на база данни с подходящ опит в проекта, която включва подобни процеси, подобни технологии, подобна среда, подобни изисквания за качество и подобни хора.
- Той е дефиниран достатъчно подробно, така че основните му рискови области да бъдат разбрани и вероятността за успех да бъде обективно оценена.

04

Теоретично- базирани форми на оценяване





Теретично-базиран модел

- Един от най-използваните методи в оценяването
 - Разглежда програмата като „връзка“ от причини
 - Процесът е изграден около „теория“
 - Включва верига от резултати от входни данни, дейности и въздействия.
-
- Включват допълнителни характеристики като предположения, рискове, влиянието на други участници
 - Опитва да оцени промяната на всеки етап от проектирането.
 - Често са представени от блок-схеми.



Кога да се използва

- Могат да се използват заедно с почти всеки друг вид оценка
- Използват във всеки сектор и с всякакъв вид работа
- Не е подходящо, когато потенциалните резултати не са известни

Основни елементи

- Използването, адаптирането и разработването.
- Събиране на информация на различни нива.
- Проучване на връзките между промените за да се потвърди или отхвърли теорията, свързваща еволюциите.

Предизвикателства

- Проект или програма няма изрична теория за промяна
- Трудно или е невъзможно да се разработи
- Лошо проектиране



05

Регресивно-базирани форми на оценяване

Регресионно - базиран модел



- Оценката на методите за оценка въз основа на исторически набори от данни изисква тези набори от данни да бъдат представителни за настоящи или бъдещи проекти.
- Наборът от данни може да се използва за извличане на информация за прогнозиране на разходите за нови проекти.
- Регресионните модели се използват като метод за оценка
- Новата функция за производство на разходи се използва за получаване на оценка на разходите за софтуер.
- Извършва се същото калибриране на производствената функция, за да се постигне максимална производителност.

Регресионно - базиран модел



- Регресионно-базираните оценки включват най-често срещаните параметрични модели за оценка.
- Статистическият регресионен анализ е най-подходящата техника за калибриране на моделите на изпълнение по начин, използван в Black-box.
- Прогнозният модел е предиктор, който когато се захранва с определена стойност на входните променливи, връща прогноза за стойността на продукцията.
- Целта на техниката е да открива и оценява потенциалните корелации между набори променливи и да ги използва, за да екстраполира към нови сценарии.



06

Инструменти за оценяване на софтуерни проекти

**Delphi
estimation**

WBS/CPM/PERT

SLIM



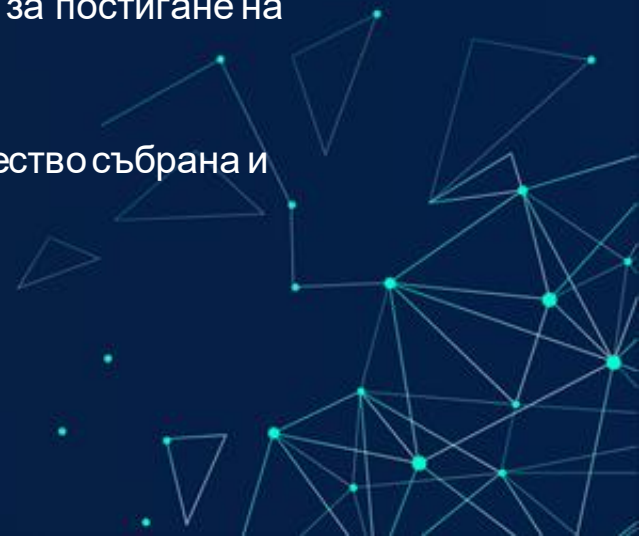
**COCOMO
модел**

**Monte-Carlo
модел**



Delphi estimation

- Известен като метод за оценка и избор
- Изисква колективна оценка, осъществена с помощта на експерти
- Анонимните мнения на експертите и анализи към тях, за постигане на еднакво мнение като резултат
- Вземане на решения, основани на определено количество събрана и анализирана информация.





Предимства

- Обоснованата оценка на няколко експерти
- Възможността за промяна на личното становище.
- Оптимизиране на процеса по събиране и анализ на информация
- Експертите работят самостоятелно
- Осигурява "поглед" напред в бъдещето

Недостатъци

- Резултати от запитването може да не оправдаят очакванията
- Значителна подготвителна работа.
- Продължителността на процедурата е неопределена.

Колко часа ще отнеме прекъсването на информационната система?



Емил

20



Григор

13



Пенчо

8

Колко часа ще отнеме прекъсването на информационната система?



Емил

20



Григор

20



Пенчо

13

Колко часа ще отнеме прекъсването на информационната система?



Приблизителна оценка

18



WBS/CPM/PERT инструменти за оценка софтуер

Съставна структурна работа (Work Breakdown Structure - WBS)



- дървовидна диаграма
- представлява разбивка на работа на по-малки задачи
- корен на структурата е самия проект, върховете са по-конкретни задания, а листата са конкретни задачи

Simple Work Breakdown Structure

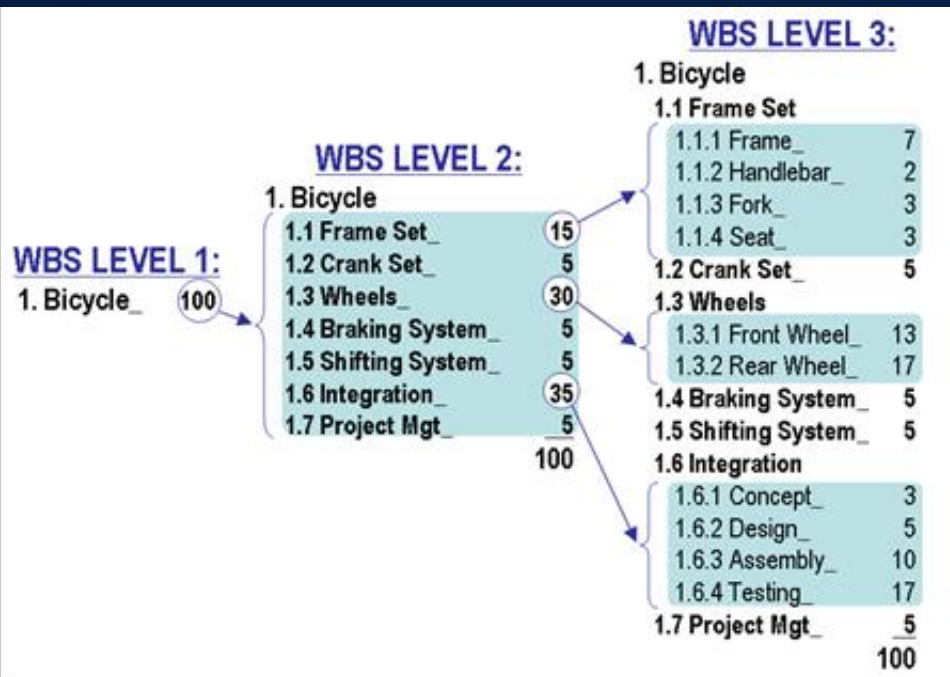


100% правило



- Важен принцип на проектиране на структурите на WBS
- WBS включва 100% от работата
- Правилото се прилага на всички нива в йерархията
- Работата, представена от дейностите във всеки работен пакет, трябва да включва до 100% от работата, необходима за завършване на работния пакет.

Пример



Метод на критичен път (Critical Path Method – CPM)



- алгоритъм, който позволява ефективното планиране и приоритизиране на съвкупността от дейности (в общ график), отчитайки времената им за изпълнение и зависимостите между тях
- критичен път в този смисъл представлява непрекъсната последователност от критични дейности, които свързват началото и края на проекта
- намира широко приложение и при планирането на използваните ресурси

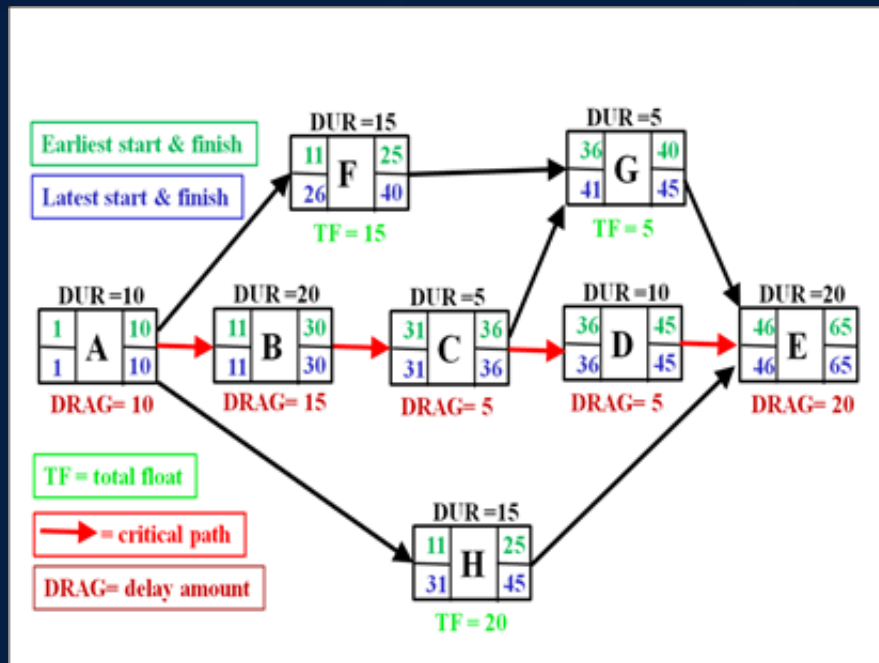


Метод на критичен път (Critical Path Method – CPM)

- техника за използване на CPM
- Този процес определя кои дейности са "критични" (т.е. по най-дългия път) и кои имат "общ поплавък" (т.е. могат да бъдат забавени, без да се удължава проектът)



Пример



Техника за преглед и оценка на програми (Program Evaluation and Review Technique – Pert)



- Техника за ефективно планиране на съвкупността от дейности (в общ график) базирана на събития и вероятности
- За да настъпи определено събитие, са необходими извършването на определени действия
- За всяко действие се дават три оценки оптимистична, песимистична и най-вероятна

Събития и дейности



PERT събитие

- предшестващо събитие
- наследяващо събитие

PERT дейност

- PERT под-дейност

Време

- оптимистично време
- песимистично време
- най-вероятно време
- очаквано време

Моделът Slim (Software Lifecycle Management)

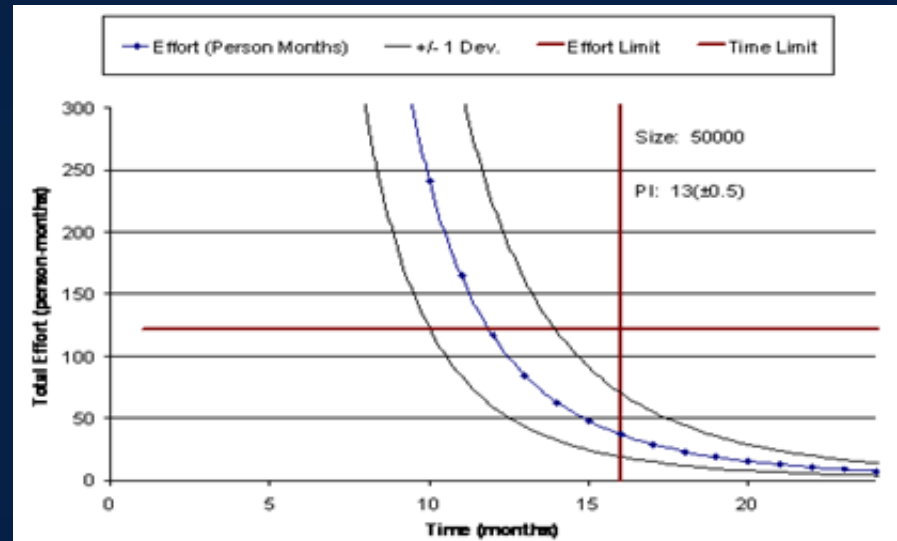


Моделът SLIM (Software Lifecycle Management) се основава на анализа на Putman за продължителността на жизнения цикъл на програмния продукт, в който зависимостта между броя редове код, производителността, времето за разработване на проекта се определя по следната формула:

$$K = (LOC / (C * t^{4/3})) * 3$$



- SLIM на Putnam е един от първите алгоритмични модели
- модел за оценка за голям проект над 70 000 реда код
- Моделът SLIM се изразява като две уравнения, описващи
- връзка между усилията за развитие и графика.
- Моделът SLIM се изразява като две уравнения, описващи връзка между усилията за развитие и графика.





COCOMO модел

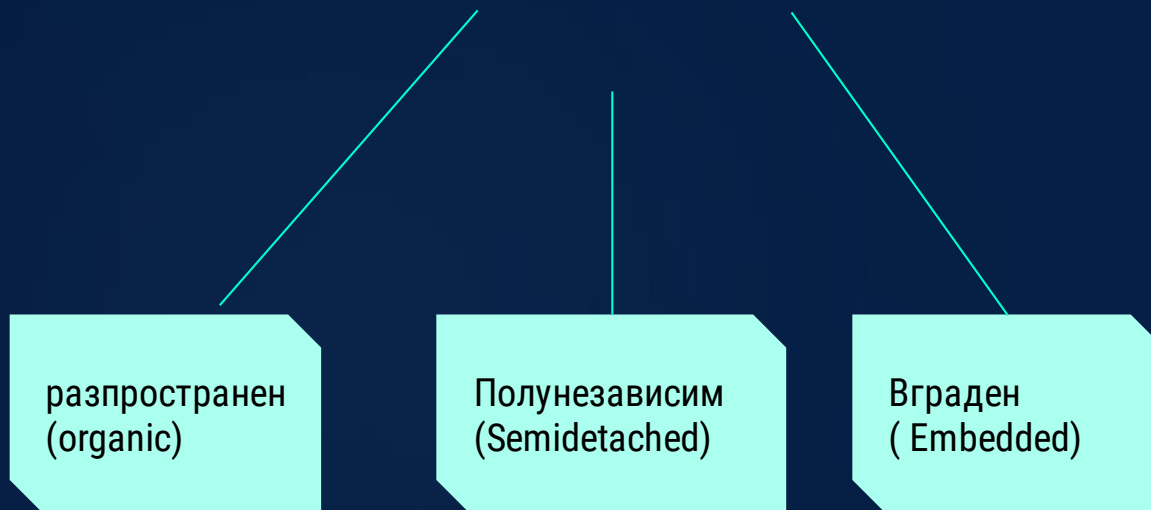
Стъпки:

- Прави се първоначална оценка на трудността на разработването от броя линии сорс код (KDLOC) Kilo Delivered Lines of Code
- Определя се набор от 15 умножаващи фактура от различни атрибути на проекта.
- Смята се усилието нужно за проекта като се умножава първоначалната оценка със всички умножаващи фактури



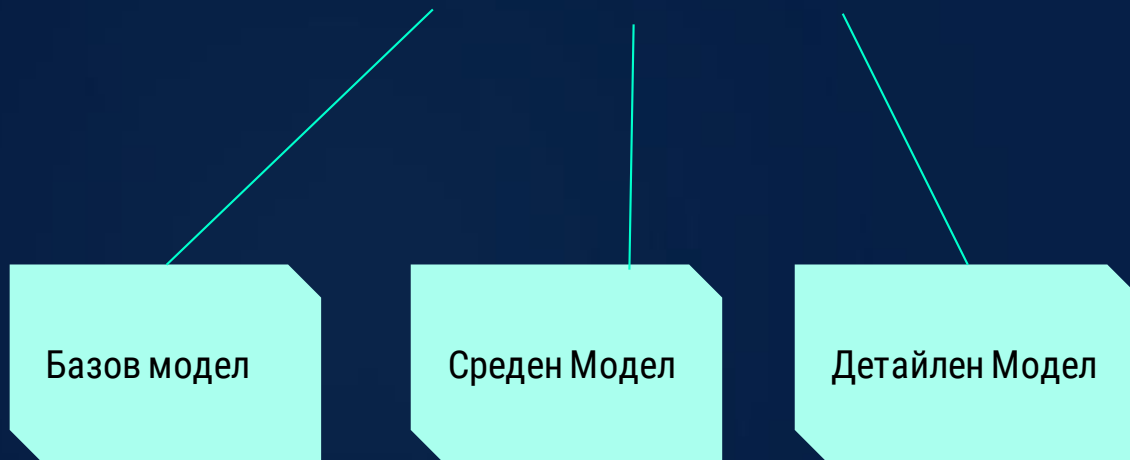


Типове проекти





Трите стадия през които преминава оценяването на софтуера



Базов модел



Уравнения:

$\text{Effort} = a_1 * (\text{KLOC})^{a_2}$ PM

$T_{\text{dev}} = b_1 * (\text{efforts})^{b_2}$ Months

За трите класове от софтуерни продукти, формулите за усилие базирани на линиите код са :

Organic: $\text{Effort} = 2.4(\text{KLOC})^{1.05}$ PM

Semi-detached: $\text{Effort} = 3.0(\text{KLOC})^{1.12}$ PM

Embedded: $\text{Effort} = 3.6(\text{KLOC})^{1.20}$ PM

Прогноза за време за разработка: формулите базирани на усилие са:

Organic: $T_{\text{dev}} = 2.5(\text{Effort})^{0.38}$ Months

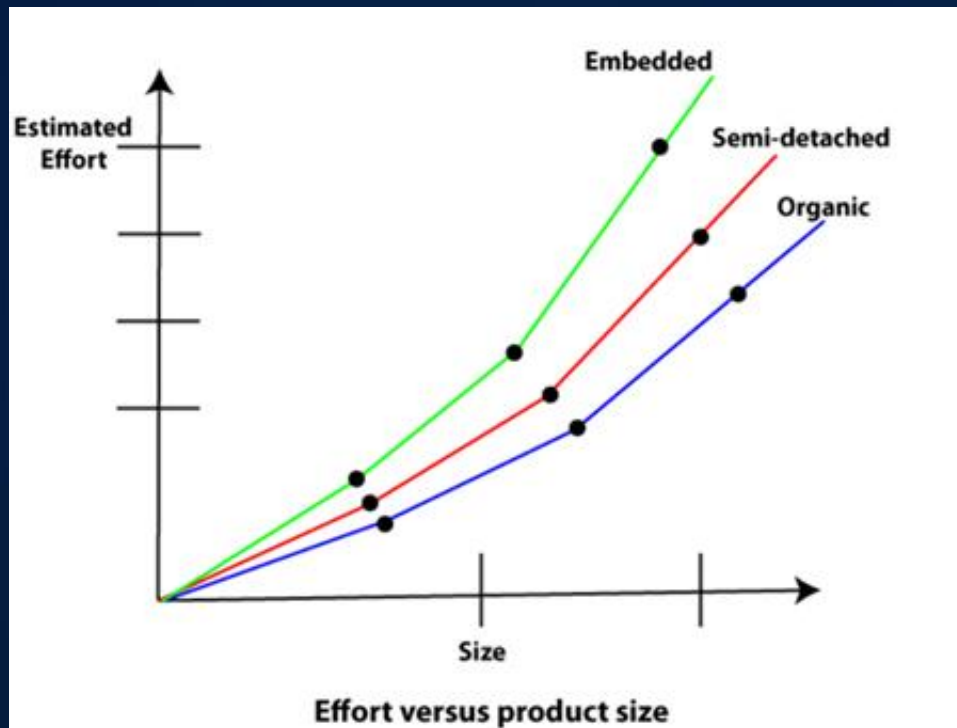
Semi-detached: $T_{\text{dev}} = 2.5(\text{Effort})^{0.35}$ Months

Embedded: $T_{\text{dev}} = 2.5(\text{Effort})^{0.32}$ Months



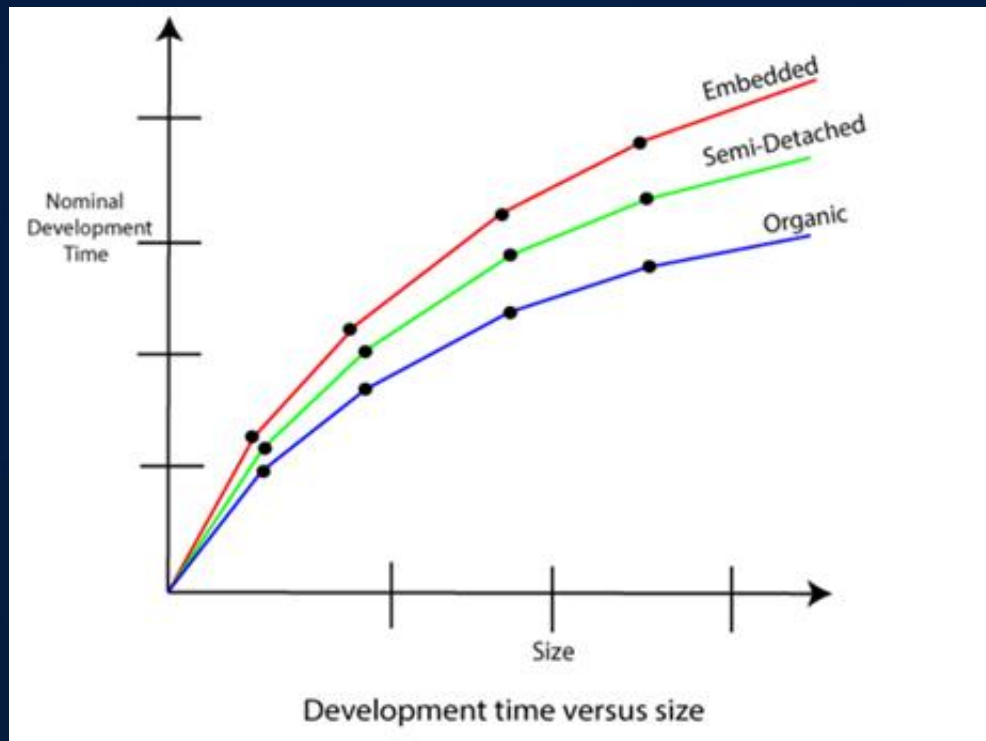


Усилието съпоставено с размера на продукта





Времето за разработка съпоставено с размера





Cosmo Казус 1:

Да предположим, че проекта е предвиден да бъде 400 KLOC. Изчислете усилието и времето за всеки от трите модела: organic, semidetached, и embedded.





Cosmo Казус 2:

Трябва да се разработи проект с размер 200 KLOC. Екипът от софтуерни инженери има среден опит с подобни проекти. Графикът не е много затегнат. Пресметнете Усилието, времето, броя на софтуерните инженери и продуктивността на проекта.





Решение на сосото казус 1

Решение : Базовото COSOMO уравнение има вида:

$$\text{Effort} = a_1 * (\text{KLOC})^{a_2} \text{ PM}$$

$$T_{\text{dev}} = b_1 * (\text{efforts})^{b_2} \text{ Months}$$

Estimated Size of project = 400 KLOC

Organic Mode:

$$E = 2.4 * (400)^{1.05} = 1295.31 \text{ PM}$$

$$D = 2.5 * (1295.31)^{0.38} = 38.07 \text{ PM}$$

Semidetached Mode:

$$E = 3.0 * (400)^{1.12} = 2462.79 \text{ PM}$$

$$D = 2.5 * (2462.79)^{0.35} = 38.45 \text{ PM}$$

Embedded Mode:

$$E = 3.6 * (400)^{1.20} = 4772.81 \text{ PM}$$

$$D = 2.5 * (4772.8)^{0.32} = 38 \text{ PM}$$





Решение на сосото казус 2

Най-добре е да използваме Semidetached Mode, като се има предвид, размера графика и опита и времето, така се получава :

$$\text{Effort} = 3.0(200)1.12 = 1133.12 \text{ PM}$$

$$T_{\text{dev}} = 2.5(1133.12)0.35 = 29.3 \text{ PM}$$

$$\text{Броя на програмистите} = \text{Effort} / T_{\text{dev}} = 1133.12 / 29.3 = 38.67 \text{ Човека}$$

$$\text{Продуктивност: } KLOC / \text{Effort} = 200 / 1133.12 = 0.1765 \text{ Kloc/PM}$$





Класификация на 15те атрибута

	Cost Drivers	RATINGS					
		Very low	Low	Nominal	High	Very High	Extra High
<u>Характеристики на продукта</u>	Product Attributes						
1. Изисквана надеждност на софтуера	RELY	0.75	0.88	1.00	1.15	1.40	..
2. Размер на базата данни на приложението	DATA	..	0.94	1.00	1.08	1.16	..
3. Сложност на продукта	CPLX	0.70	0.85	1.00	1.15	1.30	1.65
<u>Спецификации на хардуера</u>	Computer Attributes						
4. Ограничения на производителността по време на изпълнение на програмата	TIME	..	..	1.00	1.11	1.30	1.66
5. Ограничения на паметта	STOR	..	..	1.00	1.06	1.21	1.56
6. Нестабилност на средата на виртуалната машина	VIRT	..	0.87	1.00	1.15	1.30	..
7. Необходимо време за възстановяване	TURN	..	0.87	1.00	1.07	1.15	..

Характеристики на персонала

8. Аналитични умения

9. Опит за развитие

10. Способности за разработка на софтуер

11. Опит с използването на виртуални машини

12. Опит за разработка на езици за програмиране

Характеристики на проекта

13. Приложение на методи за разработка на софтуер

14. Използване на инструменти за разработка на софтуер

15. Изисквания за съответствие на графика за разработка

Cost Drivers	RATINGS					
	Very low	Low	Nominal	High	Very high	Extra high
Personnel Attributes						
ACAP	1.46	1.19	1.00	0.86	0.71	..
AEXP	1.29	1.13	1.00	0.91	0.82	..
PCAP	1.42	1.17	1.00	0.86	0.70	..
VEXP	1.21	1.10	1.00	0.90	..	..
LEXP	1.14	1.07	1.00	0.95	..	..
Project Attributes						
MODP	1.24	1.10	1.00	0.91	0.82	..
TOOL	1.24	1.10	1.00	0.91	0.83	..
SCED	1.23	1.08	1.00	1.04	1.10	..



Детайлен COSOMO модел

- Планиране и изисквания
- Системна структура
- Пълна структура
- Моделиране на ядро и тест
- Модел за конструиране на оценка





Monte-Carlo метод

Monte-Carlo методите са широк клас от изчислителни алгоритми, които разчитат на повтарянето на случайни извадки за постигане на числов резултат. Те често са използвани във физични и математически проблеми и са най-полезни, когато е трудно или невъзможно използването на други математически методи.

Те се използват в три отделни проблемни класа:

- Оптимизация;
- Числов интеграл;
- Генериране на равенства от разпределение на вероятностите.



Monte-Carlo метод



История

Дефиниции

Приложения

Инженерство

Изкуствен
интелект за
игрите

Интеграция



**Благодарим за
вниманието!**